JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class Short

java.lang.Object
    java.lang.Number
        java.lang.Short

All Implemented Interfaces:

Serializable, Comparable<Short>

```
public final class Short
extends Number
implements Comparable<Short>
```

The `Short` class wraps a value of primitive type `short` in an object. An object of type `Short` contains a single field whose type is `short`.

In addition, this class provides several methods for converting a `short` to a `String` and a `String` to a `short`, as well as other constants and methods useful when dealing with a `short`.

Since:

`JDK1.1`

See Also:

`Number`, `Serialized Form`

Field Summary

Fields

Modifier and Type	Field	Description
`static int`	**BYTES**	

		The number of bytes used to represent a `short` value in two's complement binary form.
`static short`	**`MAX_VALUE`**	A constant holding the maximum value a `short` can have, $2^{15}-1$.
`static short`	**`MIN_VALUE`**	A constant holding the minimum value a `short` can have, -2^{15}.
`static int`	**`SIZE`**	The number of bits used to represent a `short` value in two's complement binary form.
`static Class<Short>`	**`TYPE`**	The `Class` instance representing the primitive type `short`.

Constructor Summary

Constructors

Constructor	Description
`Short``(short value)`	Constructs a newly allocated `Short` object that represents the specified `short` value.
`Short``(`**`String`**` s)`	Constructs a newly allocated `Short` object that represents the `short` value indicated by the `String` parameter.

Method Summary

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
`byte`	**`byteValue`**`()`	Returns the value of this `Short` as a `byte` after a narrowing primitive conversion.

static int	**compare**(short x, short y)	Compares two short values numerically.
int	**compareTo**(**Short** anotherShort)	Compares two Short objects numerically.
static **Short**	**decode**(**String** nm)	Decodes a String into a Short.
double	**doubleValue**()	Returns the value of this Short as a double after a widening primitive conversion.
boolean	**equals**(**Object** obj)	Compares this object to the specified object.
float	**floatValue**()	Returns the value of this Short as a float after a widening primitive conversion.
int	**hashCode**()	Returns a hash code for this Short; equal to the result of invoking intValue().
static int	**hashCode**(short value)	Returns a hash code for a short value; compatible with Short.hashCode().
int	**intValue**()	Returns the value of this Short as an int after a widening primitive conversion.
long	**longValue**()	Returns the value of this Short as a long after a widening primitive conversion.

static short	**parseShort**(**String** s)	Parses the string argument as a signed decimal short.
static short	**parseShort**(**String** s, int radix)	Parses the string argument as a signed short in the radix specified by the second argument.
static short	**reverseBytes**(short i)	Returns the value obtained by reversing the order of the bytes in the two's complement representation of the specified short value.
short	**shortValue**()	Returns the value of this Short as a short.
String	**toString**()	Returns a String object representing this Short's value.
static **String**	**toString**(short s)	Returns a new String object representing the specified short.
static int	**toUnsignedInt**(short x)	Converts the argument to an int by an unsigned conversion.
static long	**toUnsignedLong**(short x)	Converts the argument to a long by an unsigned conversion.
static **Short**	**valueOf**(short s)	Returns a Short instance representing the specified short value.

static Short	valueOf(String s)	Returns a Short object holding the value given by the specified String.
static Short	valueOf(String s, int radix)	Returns a Short object holding the value extracted from the specified String when parsed with the radix given by the second argument.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Detail

MIN_VALUE

```
public static final short MIN_VALUE
```

A constant holding the minimum value a short can have, -2^{15}.

See Also:

Constant Field Values

MAX_VALUE

```
public static final short MAX_VALUE
```

A constant holding the maximum value a short can have, $2^{15}-1$.

See Also:

Constant Field Values

TYPE

```
public static final Class<Short> TYPE
```

The Class instance representing the primitive type short.

SIZE

```
public static final int SIZE
```

The number of bits used to represent a `short` value in two's complement binary form.

Since:

1.5

See Also:

Constant Field Values

BYTES

```
public static final int BYTES
```

The number of bytes used to represent a `short` value in two's complement binary form.

Since:

1.8

See Also:

Constant Field Values

Constructor Detail

Short

```
public Short(short value)
```

Constructs a newly allocated `Short` object that represents the specified `short` value.

Parameters:

`value - the value to be represented by the Short.`

Short

```
public Short(String s)
      throws NumberFormatException
```

Constructs a newly allocated `Short` object that represents the `short` value indicated by the `String` parameter. The string is converted to a `short` value in exactly the manner used by the `parseShort` method for radix 10.

Parameters:

`s - the String to be converted to a Short`

Throws:

`NumberFormatException - If the String does not contain a parsable short.`

See Also:

`parseShort(java.lang.String, int)`

Method Detail

toString

```
public static String toString(short s)
```

Returns a new `String` object representing the specified `short`. The radix is assumed to be 10.

Parameters:

`s - the short to be converted`

Returns:

`the string representation of the specified short`

See Also:

`Integer.toString(int)`

parseShort

```
public static short parseShort(String s,
                               int radix)
                        throws NumberFormatException
```

Parses the string argument as a signed `short` in the radix specified by the second argument. The characters in the string must all be digits, of the specified radix (as determined by whether `Character.digit(char, int)` returns a nonnegative value) except that the first character may be an ASCII minus sign `'-'` (`'\u002D'`) to indicate a negative value or an ASCII plus sign `'+'` (`'\u002B'`) to indicate a positive value. The resulting `short` value is returned.

An exception of type `NumberFormatException` is thrown if any of the following situations occurs:

- The first argument is `null` or is a string of length zero.
- The radix is either smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`.
- Any character of the string is not a digit of the specified radix, except that the first character may be a minus sign `'-'` (`'\u002D'`) or plus sign `'+'` (`'\u002B'`) provided that the string is longer than length 1.
- The value represented by the string is not a value of type `short`.

Parameters:

`s - the String containing the short representation to be parsed`

`radix - the radix to be used while parsing s`

Returns:

`the short represented by the string argument in the specified radix.`

Throws:

`NumberFormatException - If the String does not contain a parsable short.`

parseShort

```
public static short parseShort(String s)
                        throws NumberFormatException
```

Parses the string argument as a signed decimal `short`. The characters in the string must all be decimal digits, except that the first character may be an ASCII minus sign `'-'` (`'\u002D'`) to indicate a negative value or an ASCII plus sign `'+'` (`'\u002B'`) to indicate a positive value. The resulting `short` value is returned, exactly as if the argument and the radix 10 were given as arguments to the `parseShort(java.lang.String, int)` method.

Parameters:

`s - a String containing the short representation to be parsed`

Returns:

`the short value represented by the argument in decimal.`

Throws:

`NumberFormatException - If the string does not contain a parsable short.`

valueOf

```
public static Short valueOf(String s,
                            int radix)
                     throws NumberFormatException
```

Returns a `Short` object holding the value extracted from the specified `String` when parsed with the radix given by the second argument. The first argument is interpreted as representing a signed `short` in the radix specified by the second argument, exactly as if the argument were given to the `parseShort(java.lang.String, int)` method. The result is a `Short` object that represents the `short` value specified by the string.

In other words, this method returns a `Short` object equal to the value of:

```
new Short(Short.parseShort(s, radix))
```

Parameters:

`s - the string to be parsed`

`radix - the radix to be used in interpreting s`

Returns:

`a Short object holding the value represented by the string argument in the specified radix.`

Throws:

`NumberFormatException - If the String does not contain a parsable short.`

valueOf

```
public static Short valueOf(String s)
                     throws NumberFormatException
```

Returns a `Short` object holding the value given by the specified `String`. The argument is interpreted as representing a signed decimal `short`, exactly as if the argument were given to the `parseShort(java.lang.String)` method. The result is a `Short` object that represents the `short` value specified by the string.

In other words, this method returns a `Short` object equal to the value of:

```
new Short(Short.parseShort(s))
```

Parameters:

`s - the string to be parsed`

Returns:

a Short object holding the value represented by the string argument

Throws:

NumberFormatException - If the String does not contain a parsable short.

valueOf

```
public static Short valueOf(short s)
```

Returns a Short instance representing the specified short value. If a new Short instance is not required, this method should generally be used in preference to the constructor Short(short), as this method is likely to yield significantly better space and time performance by caching frequently requested values. This method will always cache values in the range -128 to 127, inclusive, and may cache other values outside of this range.

Parameters:

s - a short value.

Returns:

a Short instance representing s.

Since:

1.5

decode

```
public static Short decode(String nm)
                   throws NumberFormatException
```

Decodes a String into a Short. Accepts decimal, hexadecimal, and octal numbers given by the following grammar:

DecodableString:

Sign$_{opt}$ *DecimalNumeral*

Sign$_{opt}$ 0x *HexDigits*

Sign$_{opt}$ 0X *HexDigits*

Sign$_{opt}$ # *HexDigits*

Sign$_{opt}$ 0 *OctalDigits*

Sign:

-

+

DecimalNumeral, *HexDigits*, and *OctalDigits* are as defined in section 3.10.1 of *The Java™ Language Specification*, except that underscores are not accepted between digits.

The sequence of characters following an optional sign and/or radix specifier ("0x", "0X", "#", or leading zero) is parsed as by the `Short.parseShort` method with the indicated radix (10, 16, or 8). This sequence of characters must represent a positive value or a `NumberFormatException` will be thrown. The result is negated if first character of the specified `String` is the minus sign. No whitespace characters are permitted in the `String`.

Parameters:

`nm - the String to decode.`

Returns:

`a Short object holding the short value represented by nm`

Throws:

`NumberFormatException - if the String does not contain a parsable short.`

See Also:

`parseShort(java.lang.String, int)`

byteValue

```
public byte byteValue()
```

Returns the value of this `Short` as a `byte` after a narrowing primitive conversion.

Overrides:

`byteValue in class Number`

Returns:

`the numeric value represented by this object after conversion to type byte.`

See *The Java™ Language Specification*:

`5.1.3 Narrowing Primitive Conversions`

shortValue

```
public short shortValue()
```

Returns the value of this `Short` as a `short`.

Overrides:

`shortValue in class Number`

Returns:

the numeric value represented by this object after conversion to type short.

intValue

```
public int intValue()
```

Returns the value of this `Short` as an `int` after a widening primitive conversion.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type int.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

longValue

```
public long longValue()
```

Returns the value of this `Short` as a `long` after a widening primitive conversion.

Specified by:

`longValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type long.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

floatValue

```
public float floatValue()
```

Returns the value of this `Short` as a `float` after a widening primitive conversion.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type float.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

doubleValue

```
public double doubleValue()
```

Returns the value of this `Short` as a `double` after a widening primitive conversion.

Specified by:

`doubleValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type double.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

toString

```
public String toString()
```

Returns a `String` object representing this `Short`'s value. The value is converted to signed decimal representation and returned as a string, exactly as if the `short` value were given as an argument to the `toString(short)` method.

Overrides:

`toString` in class `Object`

Returns:

a string representation of the value of this object in base 10.

hashCode

```
public int hashCode()
```

Returns a hash code for this `Short`; equal to the result of invoking `intValue()`.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this Short

See Also:

Object.equals(java.lang.Object), System.identityHashCode(java.lang.Object)

hashCode

```
public static int hashCode(short value)
```

Returns a hash code for a short value; compatible with Short.hashCode().

Parameters:

value - the value to hash

Returns:

a hash code value for a short value.

Since:

1.8

equals

```
public boolean equals(Object obj)
```

Compares this object to the specified object. The result is true if and only if the argument is not null and is a Short object that contains the same short value as this object.

Overrides:

equals in class Object

Parameters:

obj - the object to compare with

Returns:

true if the objects are the same; false otherwise.

See Also:

Object.hashCode(), HashMap

compareTo

```
public int compareTo(Short anotherShort)
```

Compares two Short objects numerically.

Specified by:

compareTo in interface Comparable<Short>

Parameters:

anotherShort - the Short to be compared.

Returns:

the value 0 if this Short is equal to the argument Short; a value less than 0 if this Short is numerically less than the argument Short; and a value greater than 0 if this Short is numerically greater than the argument Short (signed comparison).

Since:

1.2

compare

```
public static int compare(short x,
                          short y)
```

Compares two `short` values numerically. The value returned is identical to what would be returned by:

```
    Short.valueOf(x).compareTo(Short.valueOf(y))
```

Parameters:

x - the first short to compare

y - the second short to compare

Returns:

the value 0 if x == y; a value less than 0 if x < y; and a value greater than 0 if x > y

Since:

1.7

reverseBytes

```
public static short reverseBytes(short i)
```

Returns the value obtained by reversing the order of the bytes in the two's complement representation of the specified `short` value.

Parameters:

i - the value whose bytes are to be reversed

Returns:

the value obtained by reversing (or, equivalently, swapping) the bytes in the specified short value.

Since:

1.5

toUnsignedInt

```
public static int toUnsignedInt(short x)
```

Converts the argument to an `int` by an unsigned conversion. In an unsigned conversion to an `int`, the high-order 16 bits of the `int` are zero and the low-order 16 bits are equal to the bits of the `short` argument. Consequently, zero and positive `short` values are mapped to a numerically equal `int` value and negative `short` values are mapped to an `int` value equal to the input plus 2^{16}.

Parameters:

`x - the value to convert to an unsigned int`

Returns:

`the argument converted to int by an unsigned conversion`

Since:

1.8

toUnsignedLong

```
public static long toUnsignedLong(short x)
```

Converts the argument to a `long` by an unsigned conversion. In an unsigned conversion to a `long`, the high-order 48 bits of the `long` are zero and the low-order 16 bits are equal to the bits of the `short` argument. Consequently, zero and positive `short` values are mapped to a numerically equal `long` value and negative `short` values are mapped to a `long` value equal to the input plus 2^{16}.

Parameters:

`x - the value to convert to an unsigned long`

Returns:

`the argument converted to long by an unsigned conversion`

Since:

```
1.8
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.